

Iplacex

INFORME EVALUACION FINAL AUTOMATIZACIÓN DE CONTENEDORES DOCKER Y K8 IPLACEX 2026

Profesor: Santiago David Neira Neira.

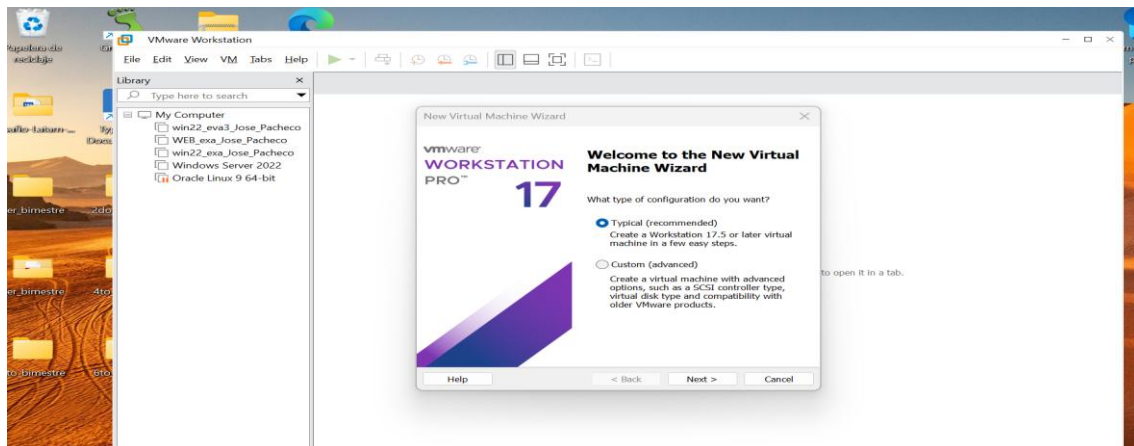
Estudiantes: José Pacheco Belisario.

Carrera: Analista programador

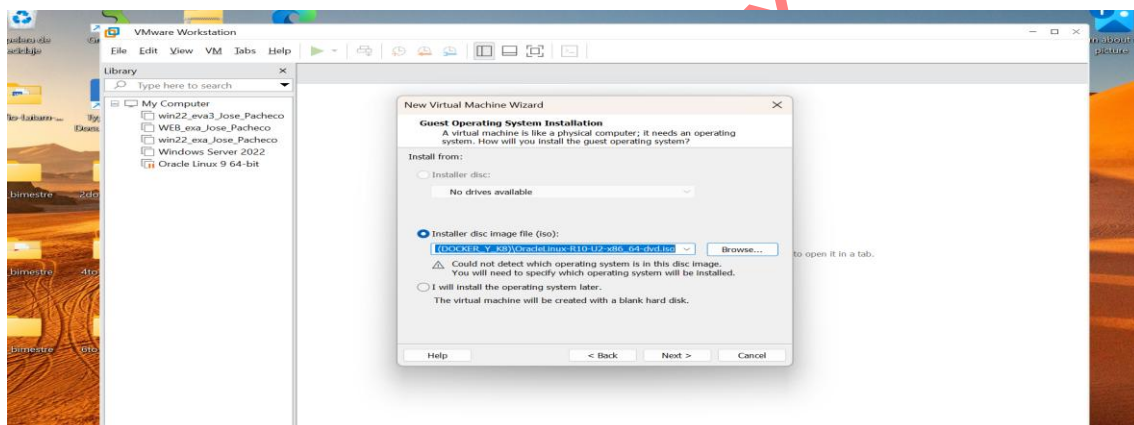
Bimestre: 3er

Año: 2026

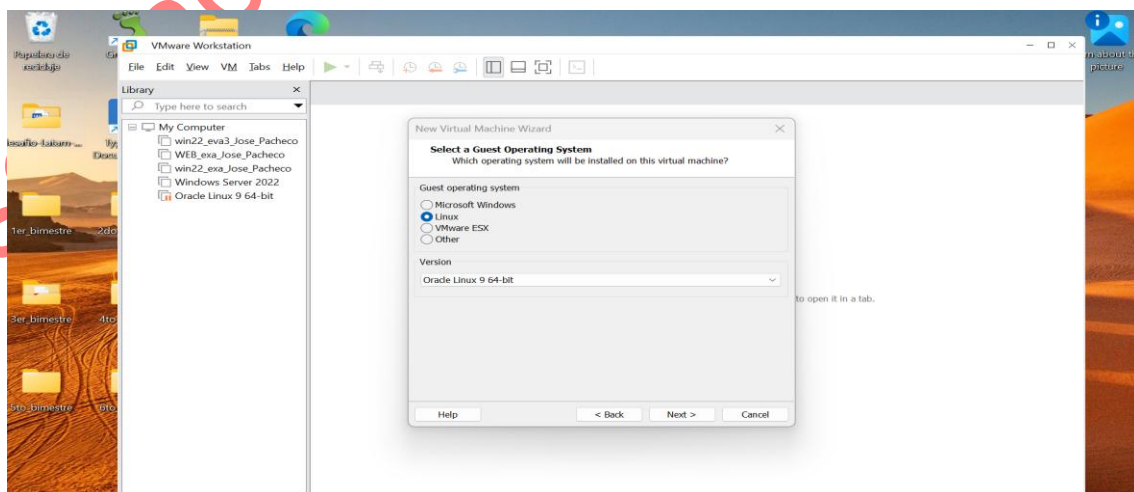
se abrió VMware Workstation 17 Pro y se seleccionó la opción para crear una nueva máquina virtual utilizando la configuración típica recomendada (Typical).



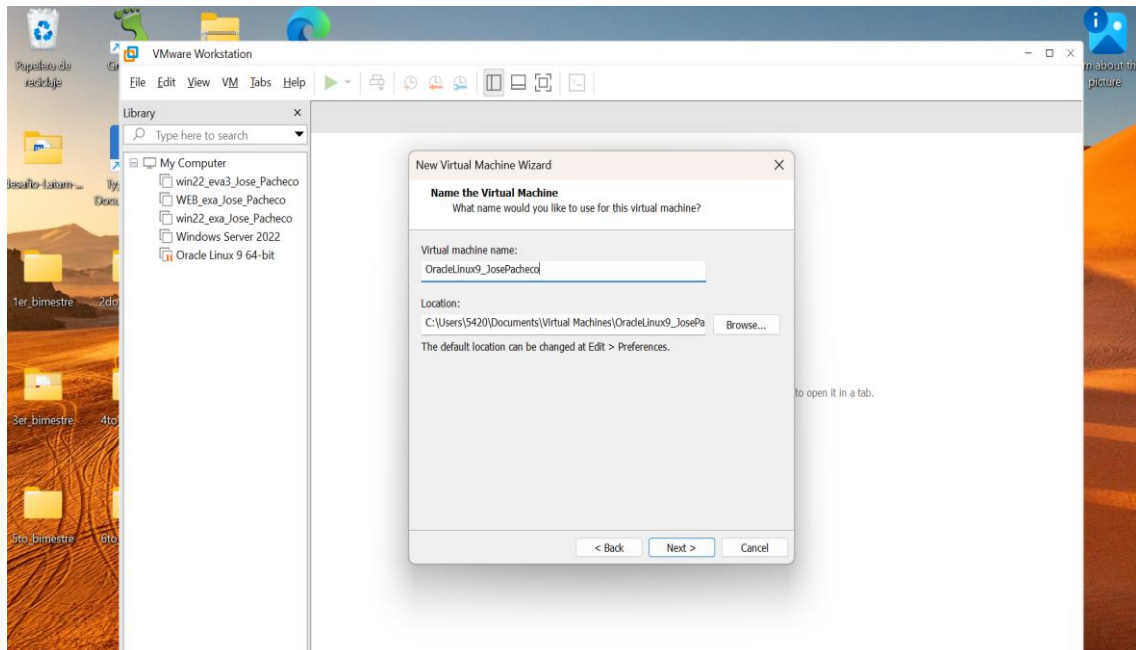
se seleccionó la opción para montar un archivo de imagen ISO (Installer disc image file) y se vinculó la ISO descargada de Oracle Linux para proceder con el despliegue del entorno.



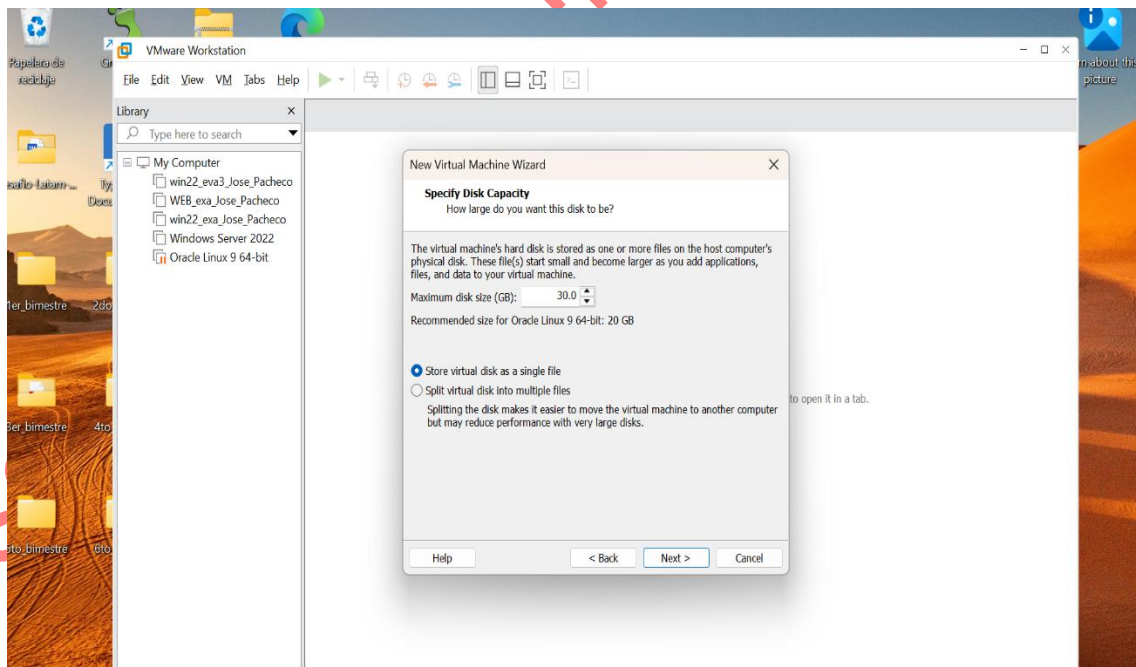
se especificó la plataforma del sistema operativo eligiendo la opción Linux y seleccionando la versión Oracle Linux 9 64-bit para la correcta compatibilidad de los controladores y recursos del hipervisor.



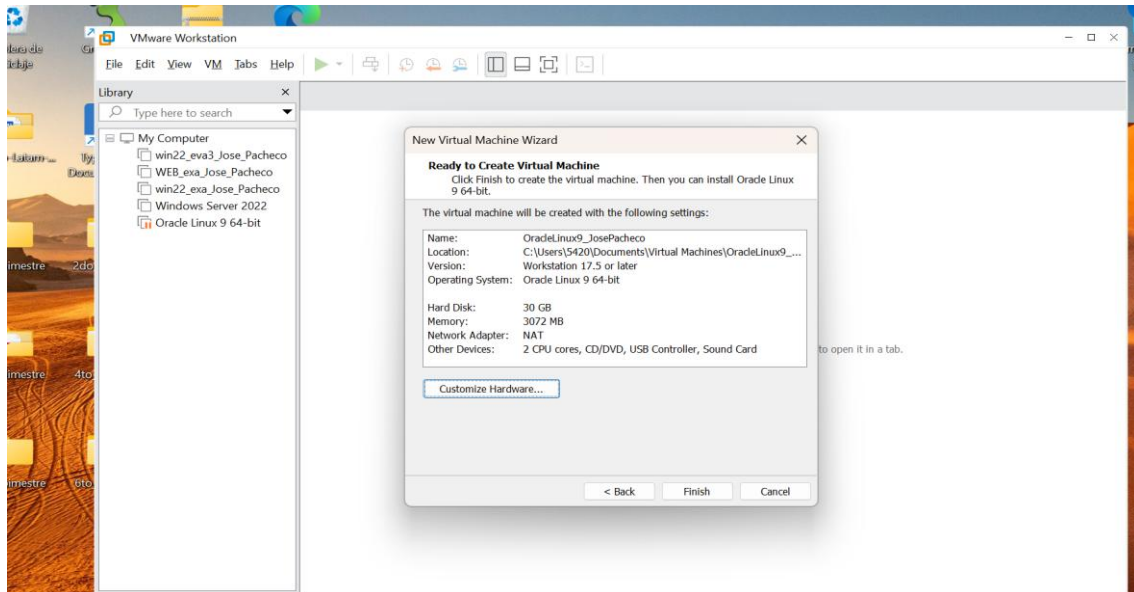
se nombró la máquina virtual como OracleLinux9_JosePacheco siguiendo los estándares de identificación del proyecto y se definió la carpeta de almacenamiento local para los archivos del entorno virtual.



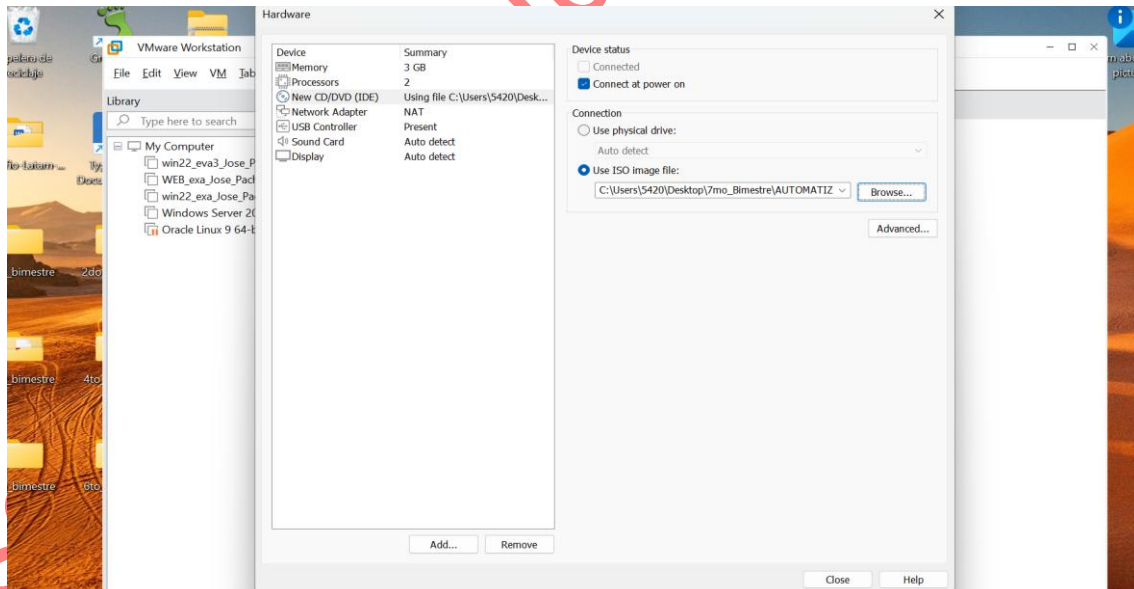
se definieron 30 GB de capacidad para el disco virtual y se eligió la opción de almacenarlo en un solo archivo (Store virtual disk as a single file) para mejorar el rendimiento del servidor.



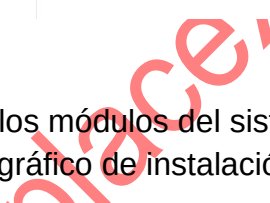
se revisó el resumen general de hardware parametrizado (Ready to Create Virtual Machine) y se confirmó la creación de la máquina virtual presionando el botón Finish.



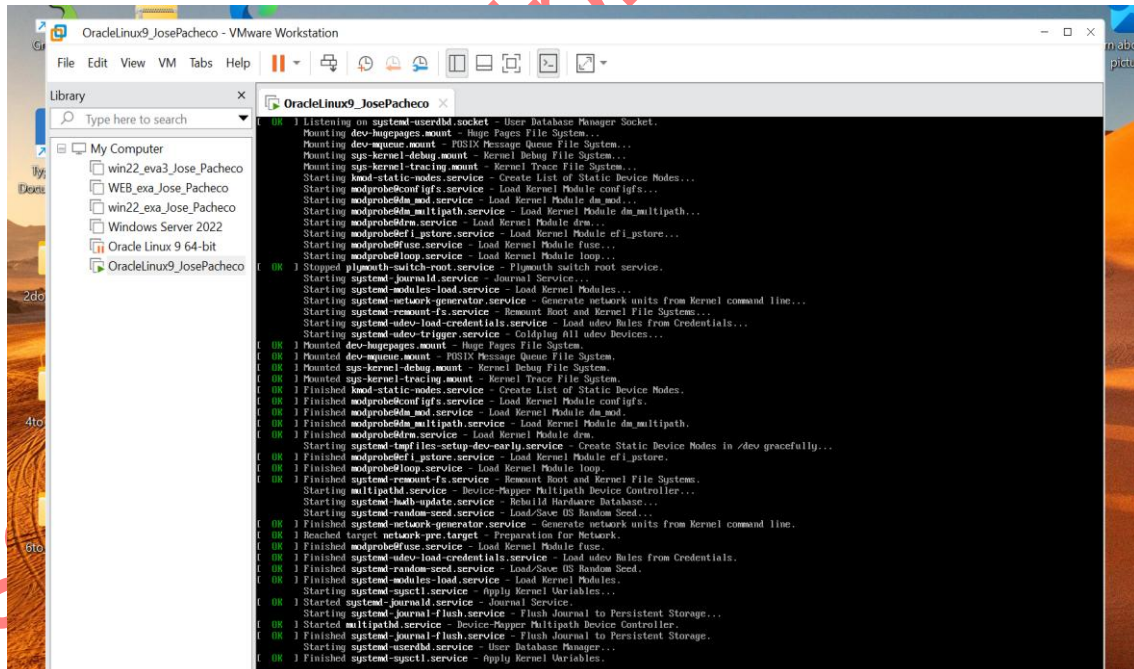
se asoció la imagen ISO de Oracle Linux 9 al lector óptico virtual (New CD/DVD) y se aseguró la opción de conexión automática al arranque (Connect at power on) para proceder con la instalación del sistema operativo.



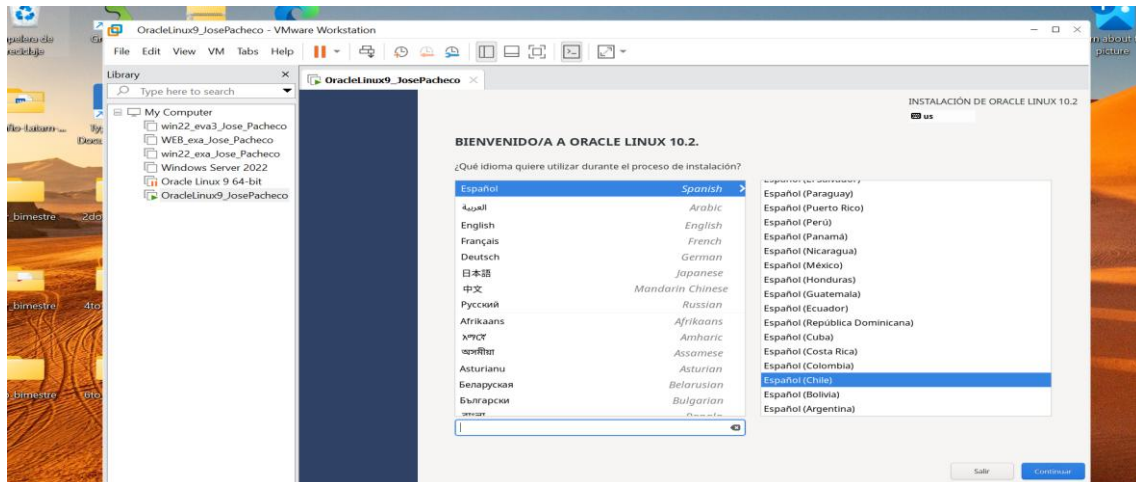
26



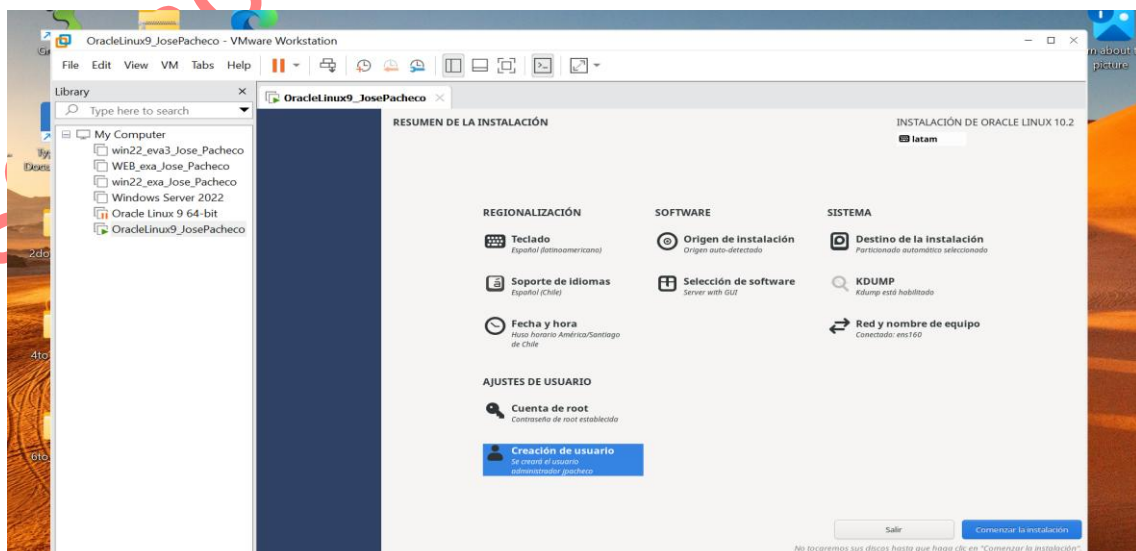
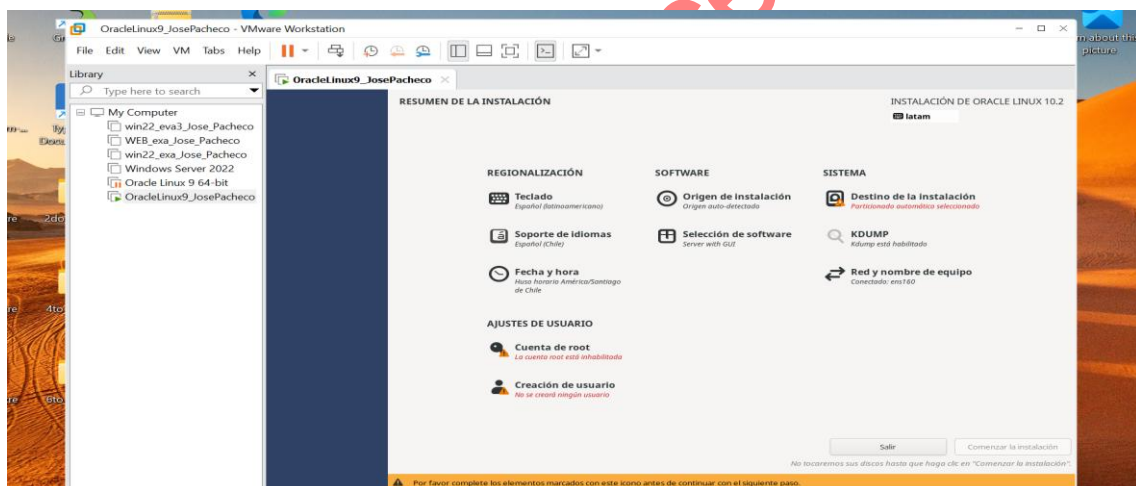
Jose



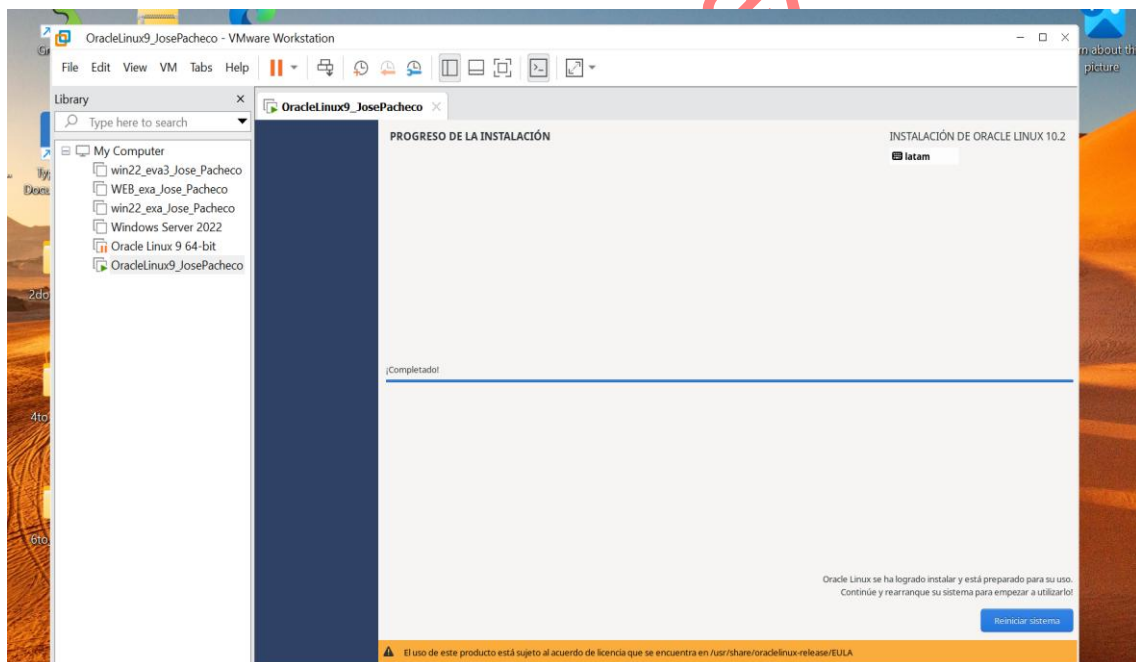
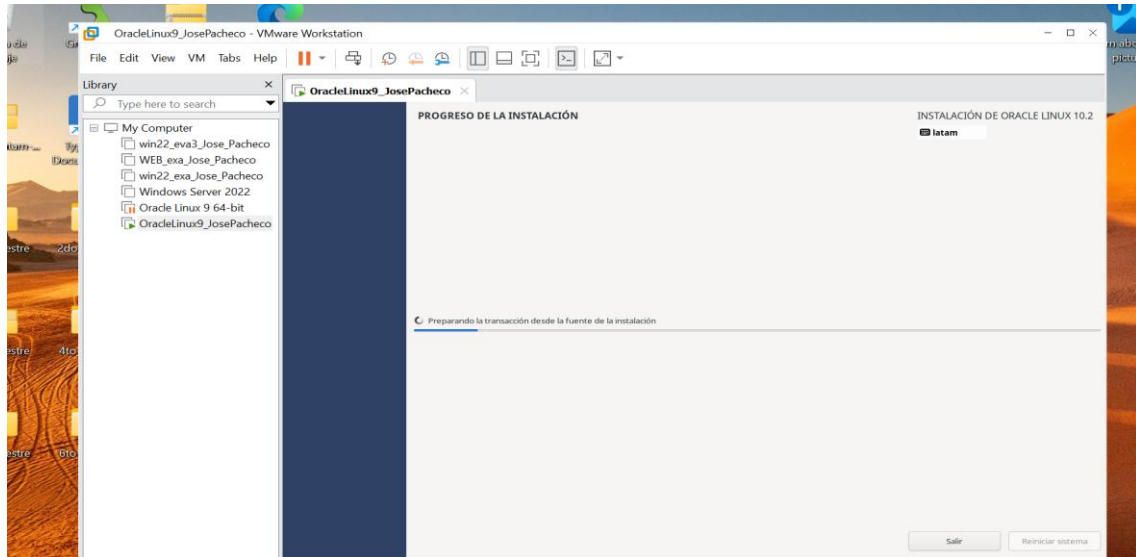
Se configuró el asistente de instalación de Oracle Linux seleccionando el idioma español y la variante regional para estructurar correctamente el teclado y las preferencias del sistema operativo.



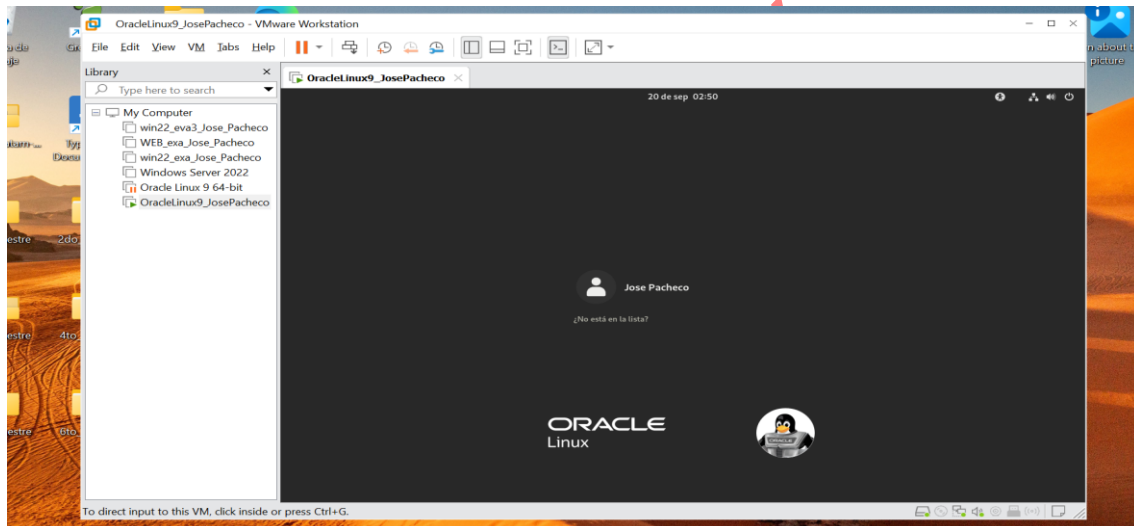
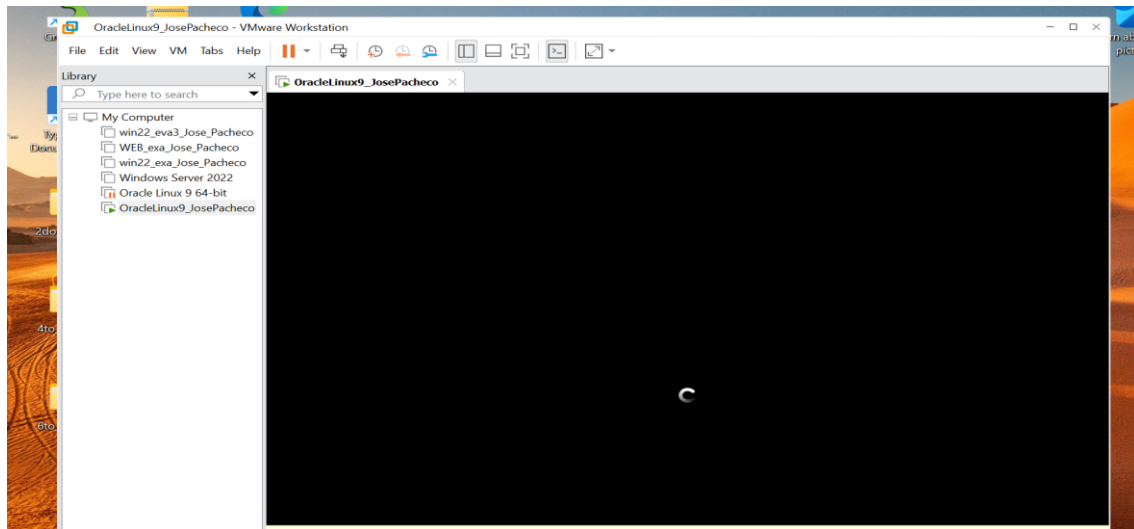
Tras confirmar la asignación del almacenamiento y definir las credenciales del sistema (cuenta root y usuario administrador jpacheco), se procedió a iniciar la instalación del sistema operativo en el disco virtual.



El proceso de instalación concluyó de manera exitosa, desplegando todos los paquetes del sistema operativo en la máquina virtual OracleLinux9_JosePacheco. Una vez completada la instalación, el asistente confirmó el estado final listo para el primer arranque.



El sistema completó el reinicio inicial e inició el entorno gráfico de Oracle Linux 9, presentando la pantalla de inicio de sesión con el usuario administrador configurado.

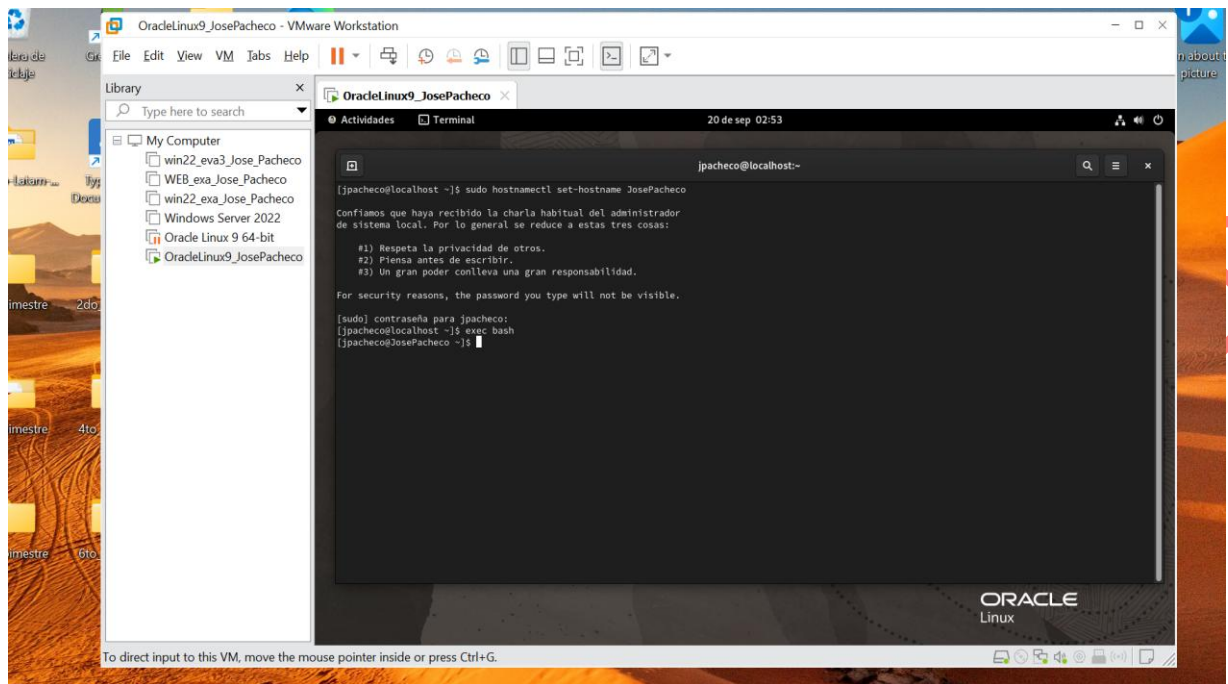


Se procedió a modificar el nombre del host (hostname) del sistema operativo para asignarle una nomenclatura personalizada.

```
sudo hostnamectl set-hostname JosePacheco
```

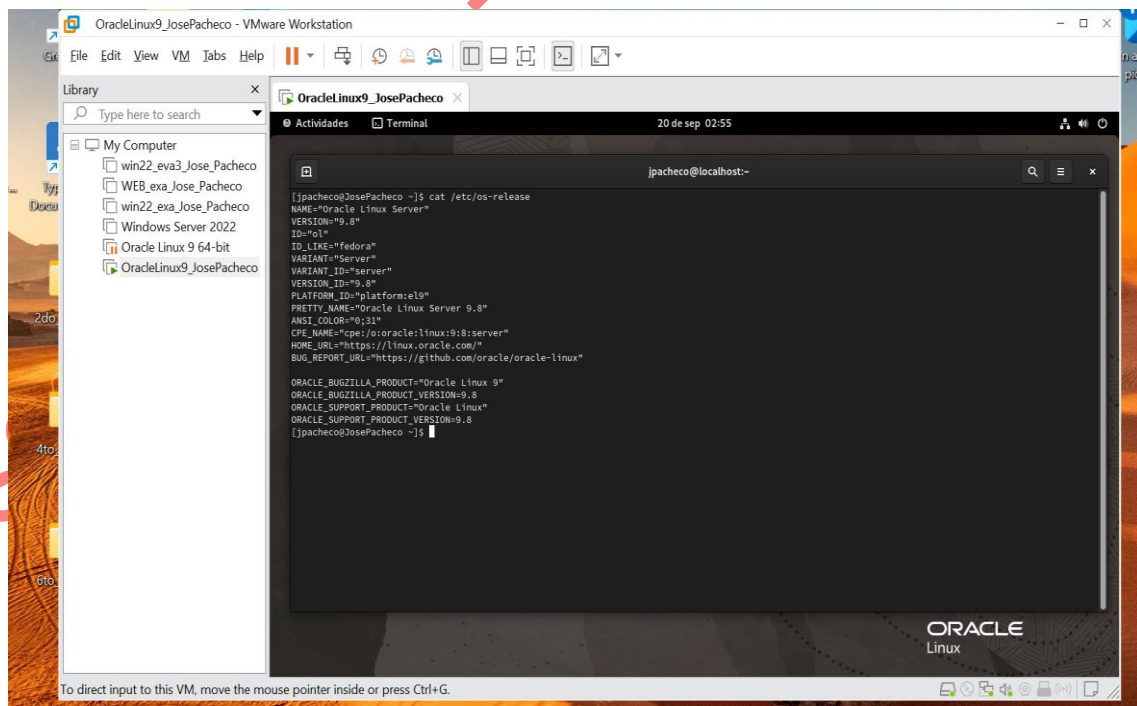
el sistema autenticó la solicitud con los privilegios del usuario administrador jpacheco. Para refrescar la instancia de la terminal y reflejar inmediatamente el cambio de jpacheco@localhost a jpacheco@JosePacheco, con este comando se refrescó la terminal.

```
exec bash
```



Al ejecutar este comando, el sistema despliega la información oficial de la distribución Oracle Linux Server (versión y metadatos), mientras que el prompt de la terminal mantiene visible la identidad del usuario y del equipo (jpacheco@JosePacheco), cumpliendo con los requerimientos de autenticidad para las evidencias del informe

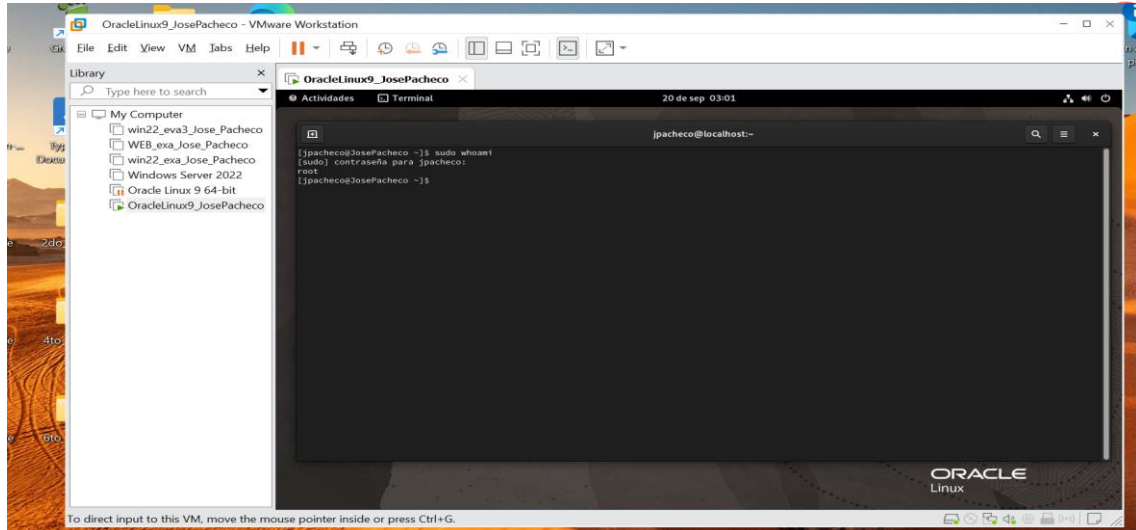
cat /etc/os-release



Con este comando verifiqué mis privilegios de administrador en el sistema.

`sudo whoami`

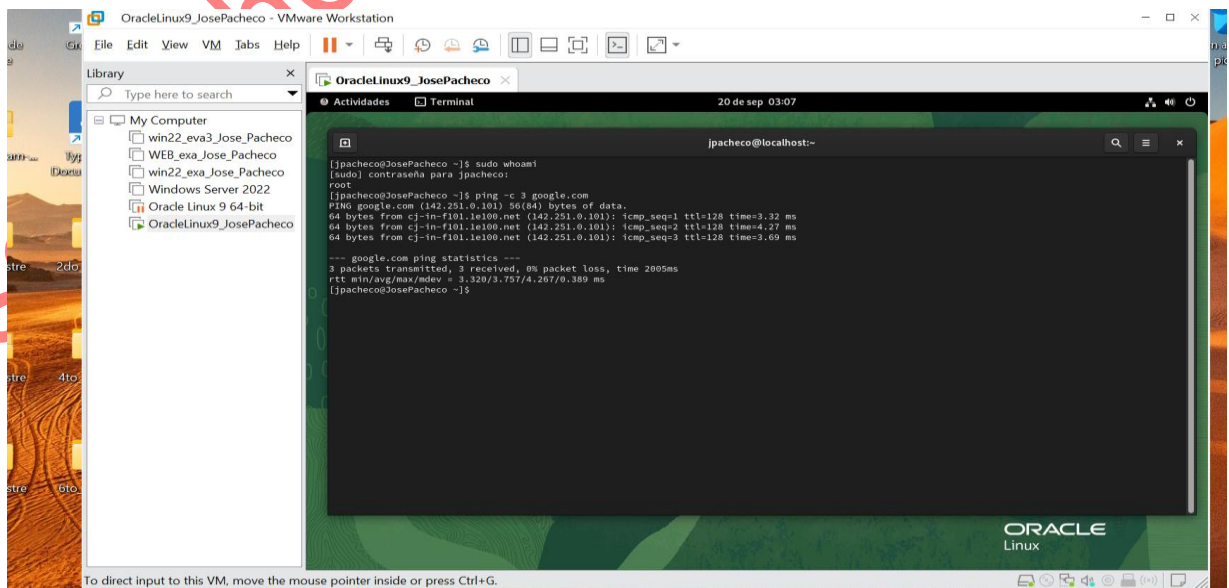
Como se observa en la captura, tras ingresar la contraseña correspondiente, el sistema devolvió el resultado `root`, confirmando que el usuario `jpacheco` tiene acceso administrativo completo para realizar las instalaciones de Docker y Kubernetes.



Con este comando verifiqué la conectividad a Internet de la máquina virtual.

`ping -c 3 google.com`

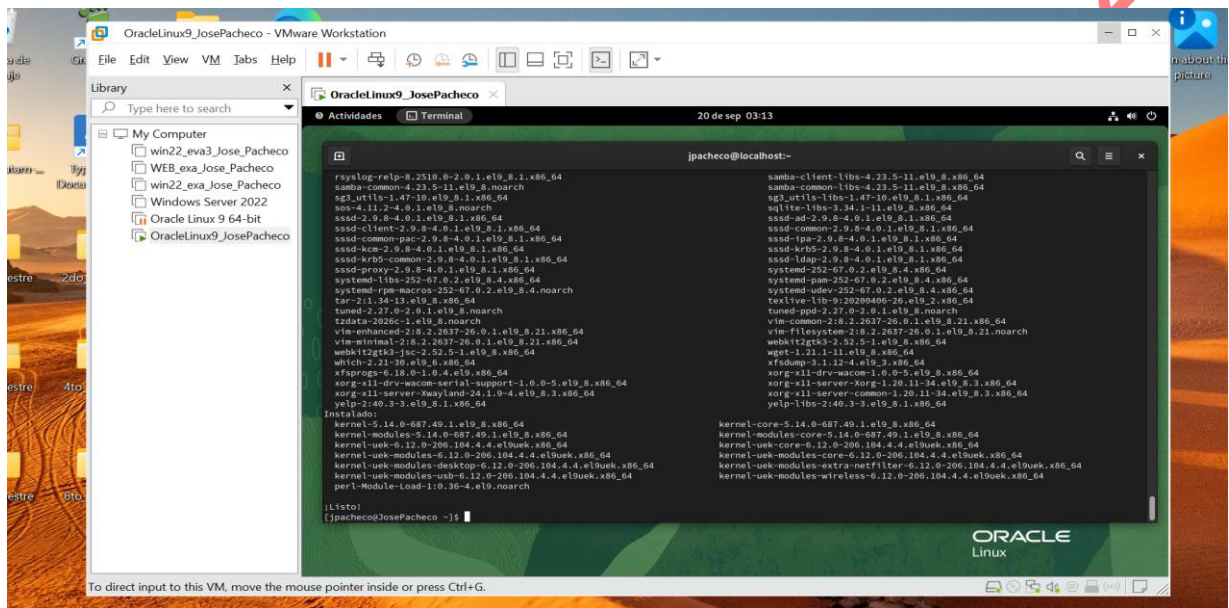
Como se observa en la captura, se transmitieron y recibieron correctamente los 3 paquetes (0% packet loss) con tiempos de respuesta óptimos (promedio de ~3.75 ms), lo que confirma que la resolución de nombres DNS y la interfaz de red funcionan correctamente para descargar dependencias.



Para garantizar que el sistema operativo cuente con las últimas versiones de seguridad, parches del kernel y librerías base antes de instalar la infraestructura de contenedores, se ejecutó la actualización general del sistema.

```
sudo dnf update -y
```

El gestor DNF procesó la descarga y actualización de las dependencias, asegurando un entorno totalmente estable y libre de vulnerabilidades para los siguientes pasos.



Para llevar a cabo la instalación del motor de contenedores, se ejecutaron los siguientes comandos secuenciales.

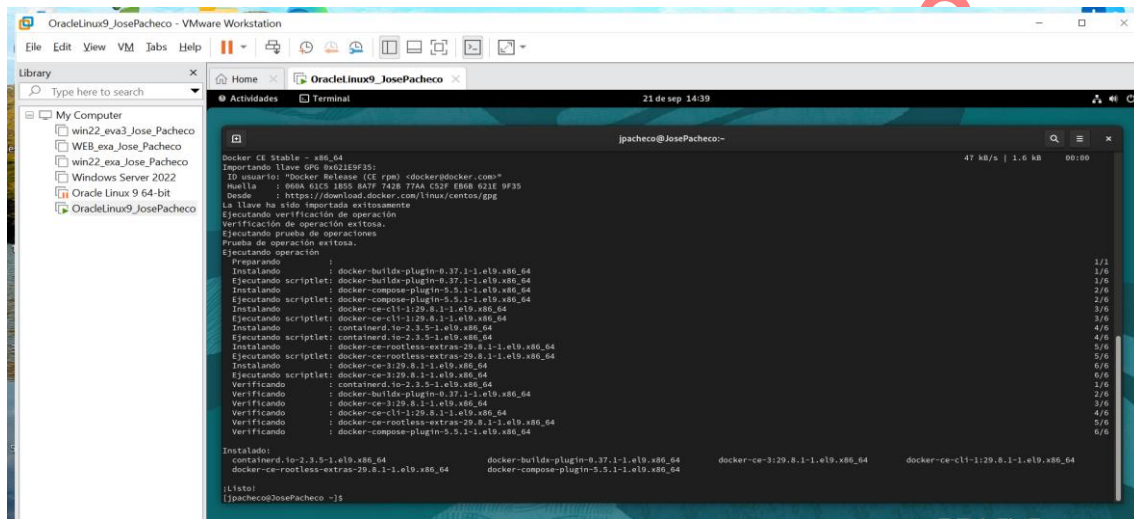
```
sudo dnf install -y dnf-plugins-core
```

```
sudo dnf config-manager --add-repo
```

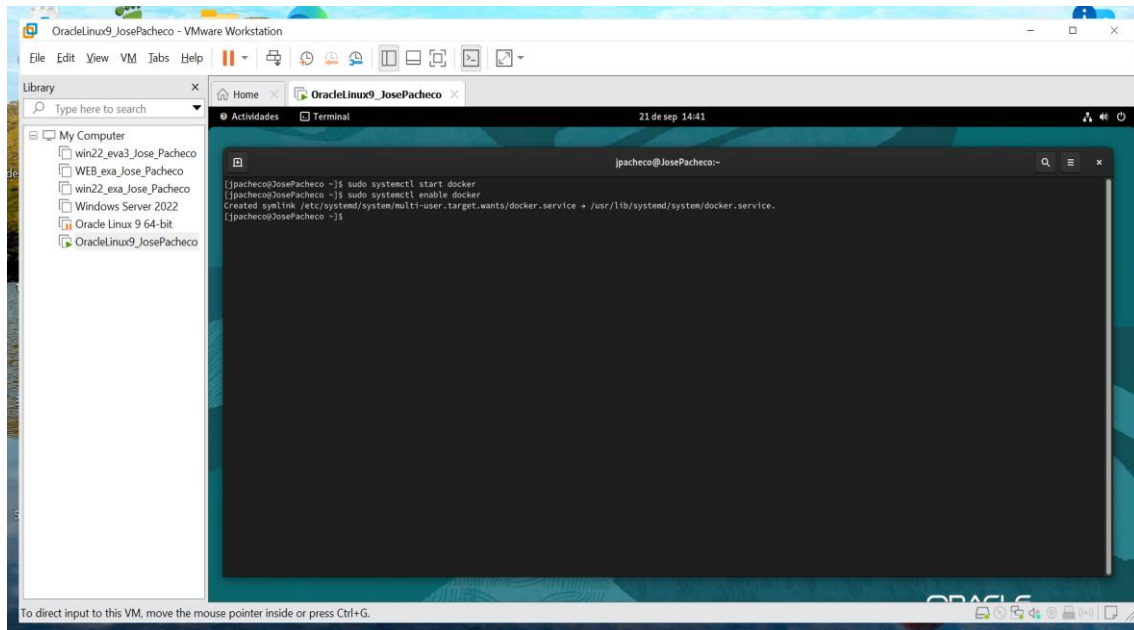
<https://download.docker.com/linux/centos/docker-ce.repo>

```
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin  
docker-compose-plugin
```

Como se observa en la captura, el sistema resolvió las dependencias correctamente y procedió a descargar e instalar los 6 paquetes esenciales (docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin, docker-compose-plugin y docker-ce-rootless-extras).



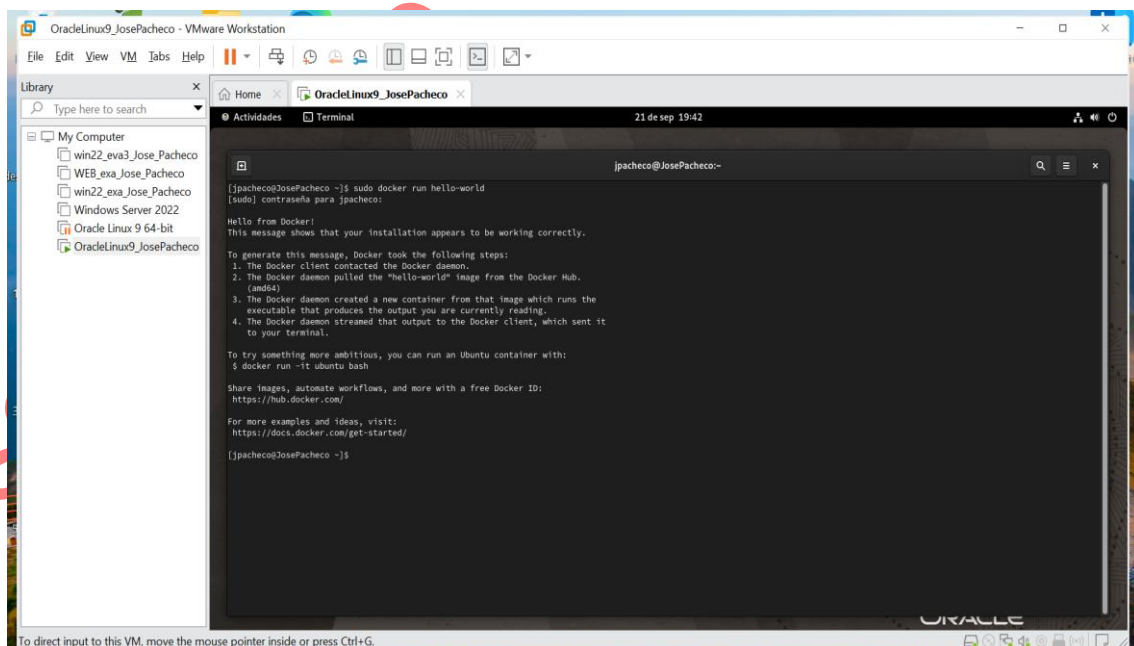
Como se evidencia en la captura, el comando creó exitosamente el enlace Simbólico (symlink) necesario en `/etc/systemd/system/multiuser.target.wants/docker.service`, confirmando que el servicio quedó registrado de forma permanente.



Para validar que la instalación de Docker está operativa y funcional, se ejecutó el siguiente comando.

```
sudo docker run hello-world
```

Como se aprecia en la captura, el motor descargó la imagen hello-world desde Docker Hub y la ejecutó en un contenedor temporal, mostrando el mensaje Hello from Docker!



Para mantener las buenas prácticas de administración en el motor de contenedores y evitar el consumo innecesario de almacenamiento, se realizaron las tareas de inspección y limpieza del sistema.

Listar todos los contenedores.

```
sudo docker ps -a
```

Eliminar de forma masiva los contenedores derivados de hello-world.

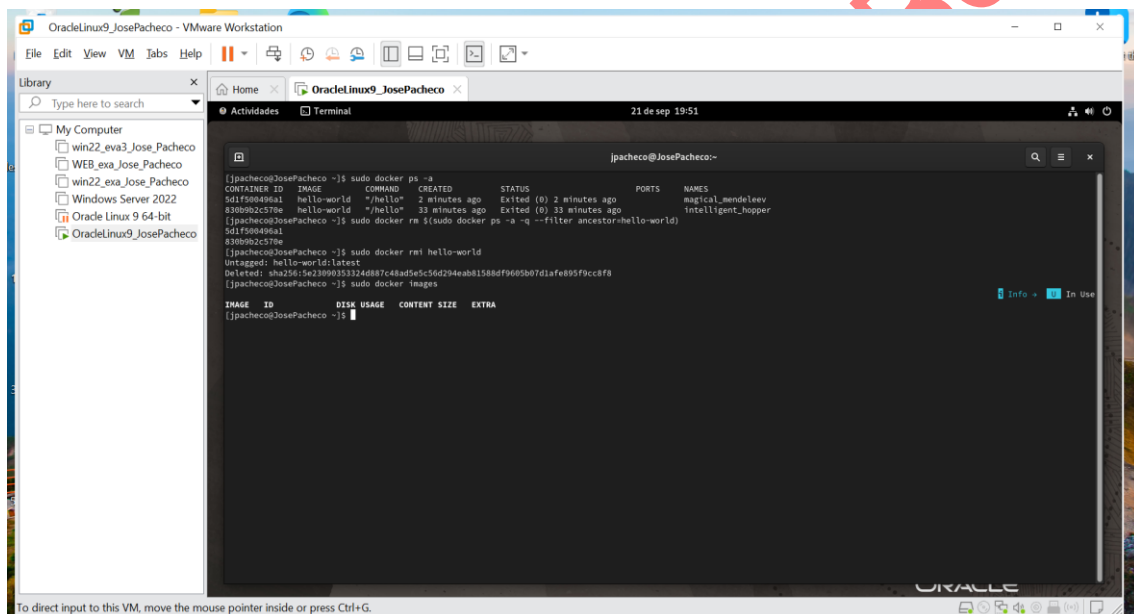
```
sudo docker rm $(sudo docker ps -a -q --filter ancestor=hello-world)
```

Eliminar la imagen base descargada.

```
sudo docker rmi hello-world
```

Listar las imágenes locales para confirmar la eliminación.

```
sudo docker images
```



Con la finalidad de preparar los archivos estáticos que conformarán el sitio web dentro del contenedor, se realizaron las siguientes operaciones desde la terminal.

Descargar el archivo comprimido con el código fuente de la aplicación.

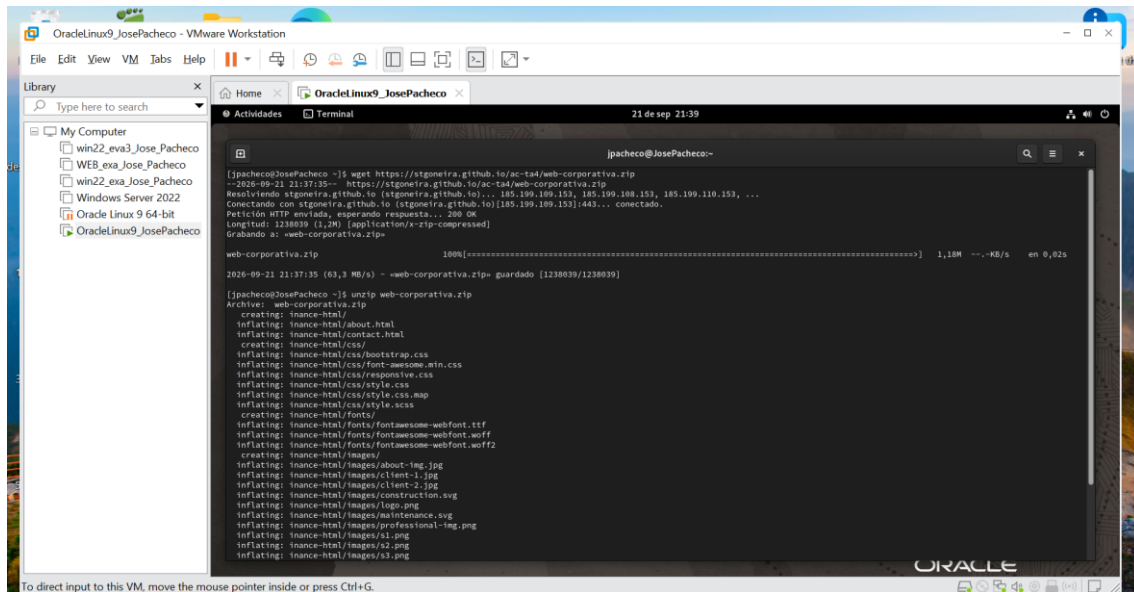
```
wget https://stgoneira.github.io/ac-ta4/web-corporativa.zip
```

Descomprimir el contenido del proyecto.

```
unzip web-corporativa.zip
```

Como se evidencia en la captura, el archivo web-corporativa.zip (de aproximadamente 1.2 MB) fue descargado exitosamente. Posteriormente, al ejecutar la descompresión, se generó la estructura de directorios dentro de la carpeta inance-html/, la cual contiene las plantillas HTML (about.html,

contact.html), las hojas de estilo CSS, las fuentes web y los recursos multimedia correspondientes al proyecto.



Para asegurar que todos los componentes requeridos por la aplicación web se encuentren presentes en la ruta de trabajo local, se ejecutaron las siguientes instrucciones.

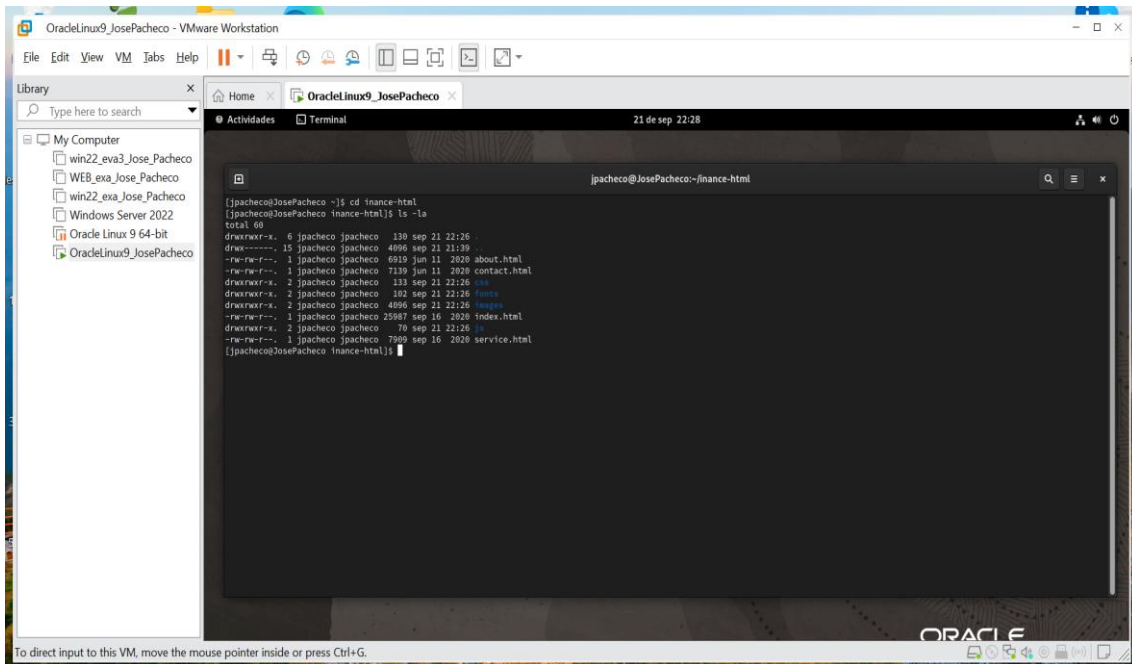
Acceder al directorio extraído.

```
cd inance-html
```

Listar todos los archivos y directorios con sus detalles de permisos y propietario.

```
ls -la
```

Como se observa en la captura, se verificó la existencia de los archivos clave del sitio estático: index.html, about.html, contact.html, service.html, junto con los subdirectorios css/, fonts/, images/ y js/. Todos los elementos pertenecen al usuario jpacheco y poseen los permisos de lectura adecuados para ser copiados posteriormente dentro de la imagen del contenedor web.



Para automatizar el empaquetado y despliegue del sitio web corporativo, se creó el archivo de especificación Dockerfile mediante el editor nano. Dentro del archivo se ingresó la siguiente configuración:

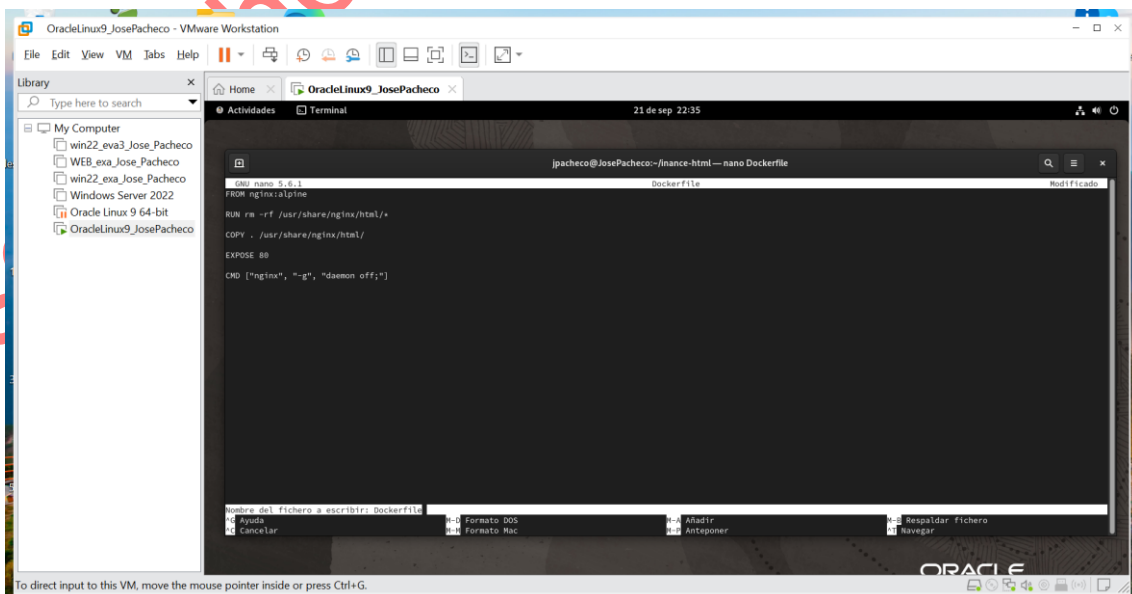
FROM nginx:alpine

RUN rm -rf /usr/share/nginx/html/*

COPY . /usr/share/nginx/html/

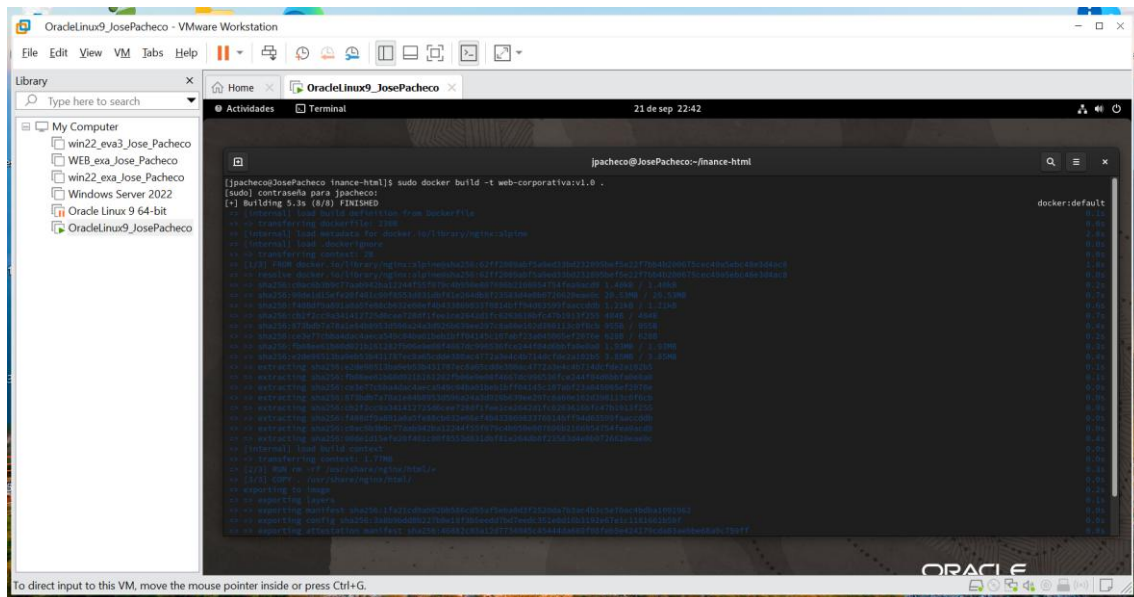
EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]



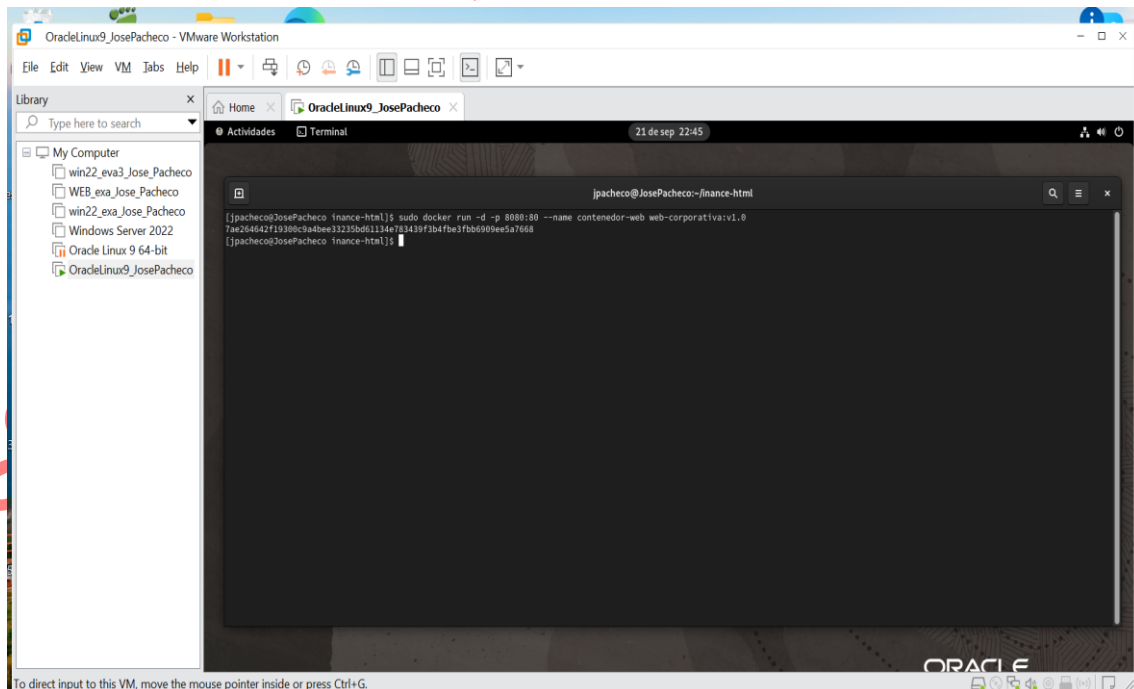
Para compilar y empaquetar la aplicación web dentro de un contenedor reutilizable, se ejecutó el siguiente comando en la terminal.

`sudo docker build -t web-corporativa:v1.0 .`



Para validar la correcta ejecución de la imagen y asegurar que el servidor Nginx sirve la aplicación adecuadamente, se instanció el contenedor mediante el siguiente comando.

`sudo docker run -d -p 8080:80 --name contenedor-web web-corporativa:v1.0`

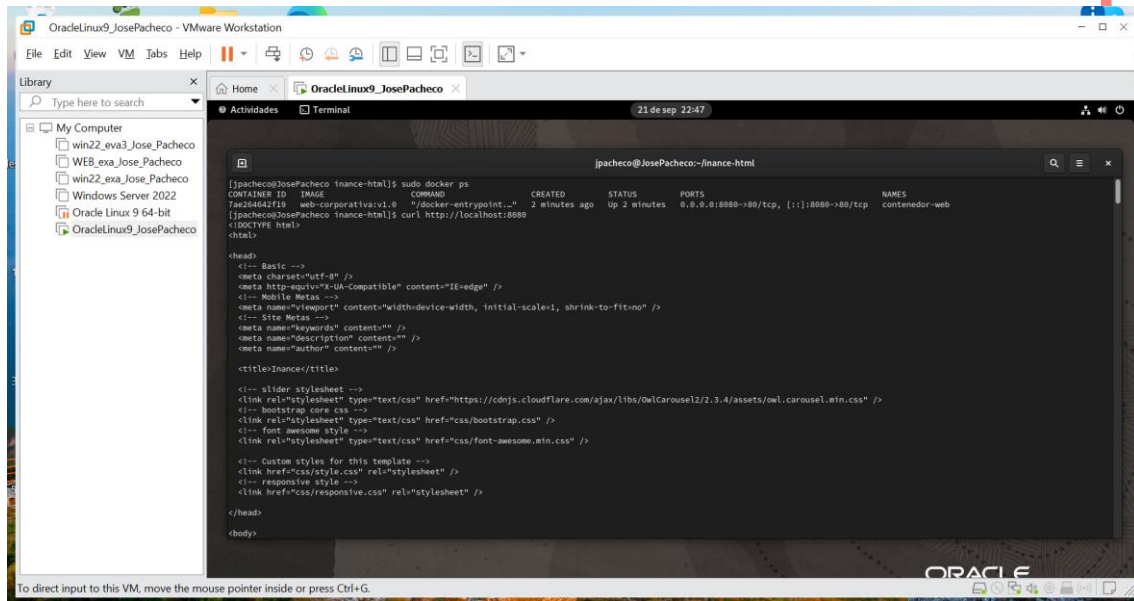


Para validar la disponibilidad operativa del servicio y confirmar la correcta redirección de puertos, se ejecutaron las comprobaciones del estado del servicio.

sudo docker ps

Probar la respuesta del servidor web

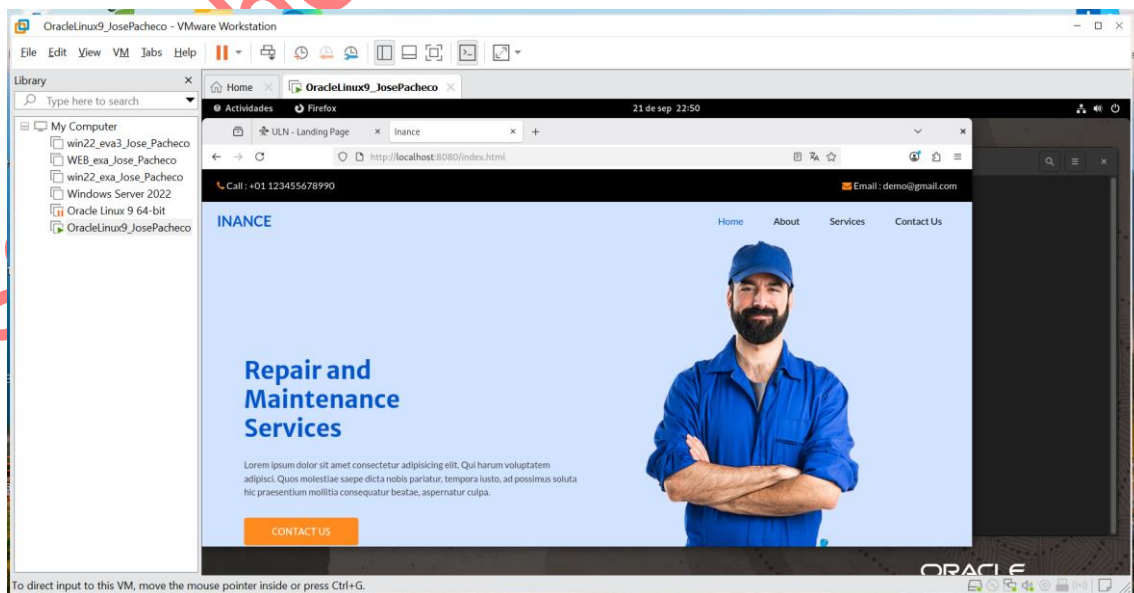
curl <http://localhost:8080>



```
jpacheco@JosePacheco--finance.html
[jpacheco@JosePacheco inance-html]$ sudo docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
7ae264642f13   web-corporativa:v1.0   "/docker-entrypoint..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   contenedor-web
[jpacheco@JosePacheco inance-html]$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<!-- Basic -->
<meta charset="utf-8" />
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<!-- Mobile Metas -->
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no" />
<!-- Site Metas -->
<meta name="keyword" content="" />
<meta name="description" content="" />
<meta name="author" content="" />
<title>Inance</title>
<!-- slider stylesheet -->
<link rel="stylesheet" type="text/css" href="https://cdnjs.cloudflare.com/ajax/libs/OwlCarousel2/2.3.4/assets/owl.carousel.min.css" />
<!-- bootstrap core css -->
<link rel="stylesheet" type="text/css" href="css/bootstrap.css" />
<!-- font awesome style -->
<link rel="stylesheet" type="text/css" href="css/font-awesome.min.css" />
<!-- Custom styles for this template -->
<link href="css/style.css" rel="stylesheet" />
<!-- responsive style -->
<link href="css/responsive.css" rel="stylesheet" />
</head>
<body>
```

Para validar la correcta carga de los archivos multimedia, estilos CSS y scripts de la plantilla, se accedió vía navegador web a la dirección mapeada del contenedor.

<http://localhost:8080>



Para definir la infraestructura como código (IaC) y orquestar la aplicación dentro del clúster de Kubernetes, se creó el manifiesto de despliegue mediante el editor de texto nano.

```
nano deployment-web.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: deployment-web-corporativa
```

```
  labels:
```

```
    app: web-corporativa
```

```
    entorno: produccion
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: web-corporativa
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: web-corporativa
```

```
        version: v1.0
```

```
    spec:
```

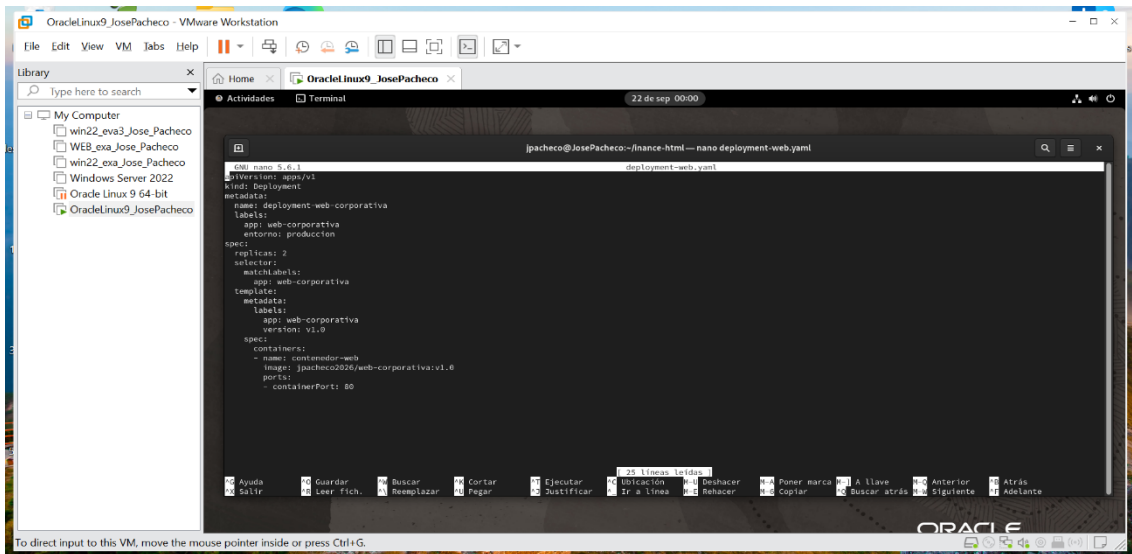
```
      containers:
```

```
        - name: contenedor-web
```

```
          image: jpacheco2026/web-corporativa:v1.0
```

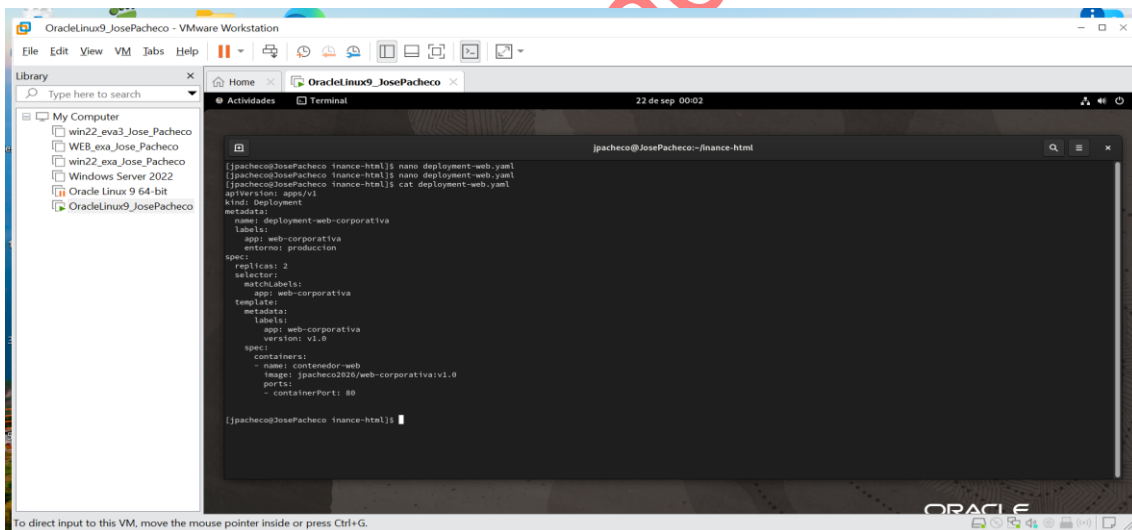
```
          ports:
```

- containerPort: 80



Para garantizar la integridad del archivo de configuración antes del despliegue, se ejecutó una inspección del manifiesto en la terminal.

cat deployment-web.yaml

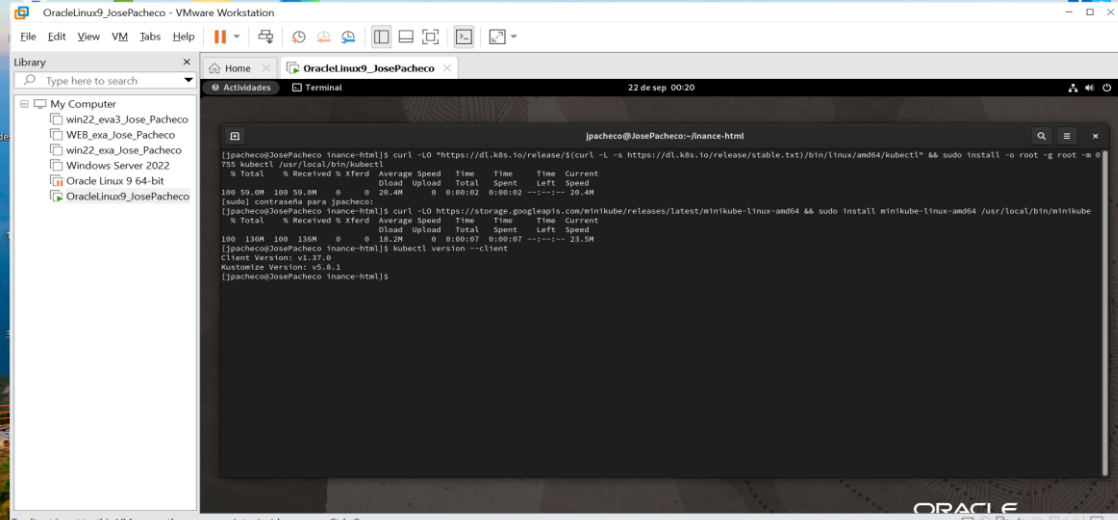


Para habilitar la gestión local de Kubernetes, se procedió con la instalación de las utilidades kubectl y minikube ejecutando las instrucciones oficiales de descarga e instalación de binarios.

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" && sudo install -o
root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-
linux-amd64 && sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

kubectl version --client

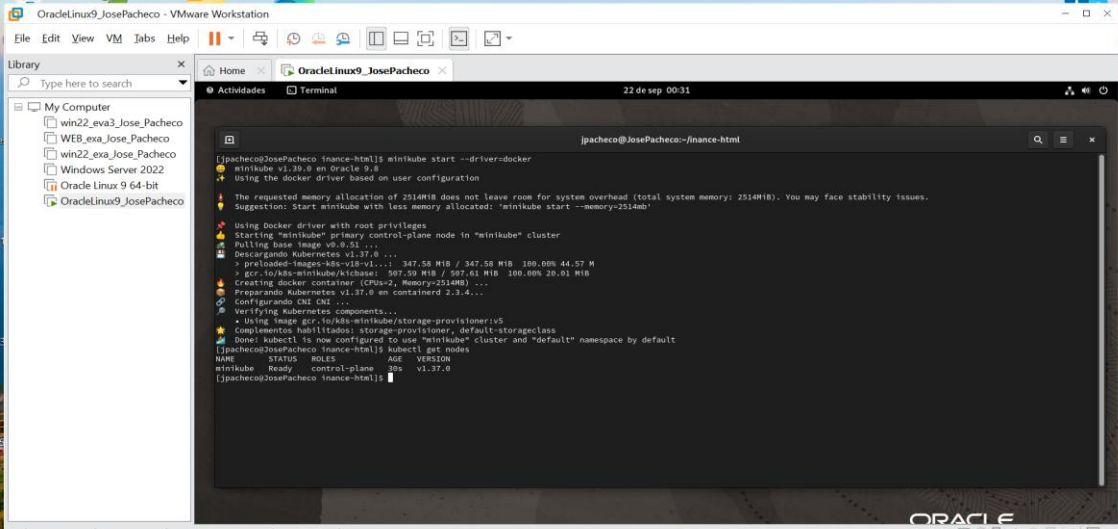


```
[jpacheco@JosePacheco ~]$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" && sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 59.0M 100 59.0M 0 0 29.0M 0 0 0 0 0 0 0 0 0 0
[sudo] contraseña para jpacheco:
[jpacheco@JosePacheco ~]$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64 && sudo install minikube-linux-amd64 /usr/local/bin/minikube
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 136M 100 136M 0 0 19.2M 0 0 0 0 0 0 0 0 0 0
[jpacheco@JosePacheco ~]$ kubectl version --client
Client Version: v1.37.0
Kustomize Version: v5.8.1
[jpacheco@JosePacheco ~]$
```

Para iniciar el clúster mono-nodo de Kubernetes, se ejecutó la orden de arranque especificada para el controlador de Docker.

`minikube start --driver=docker`

`kubectl get nodes`



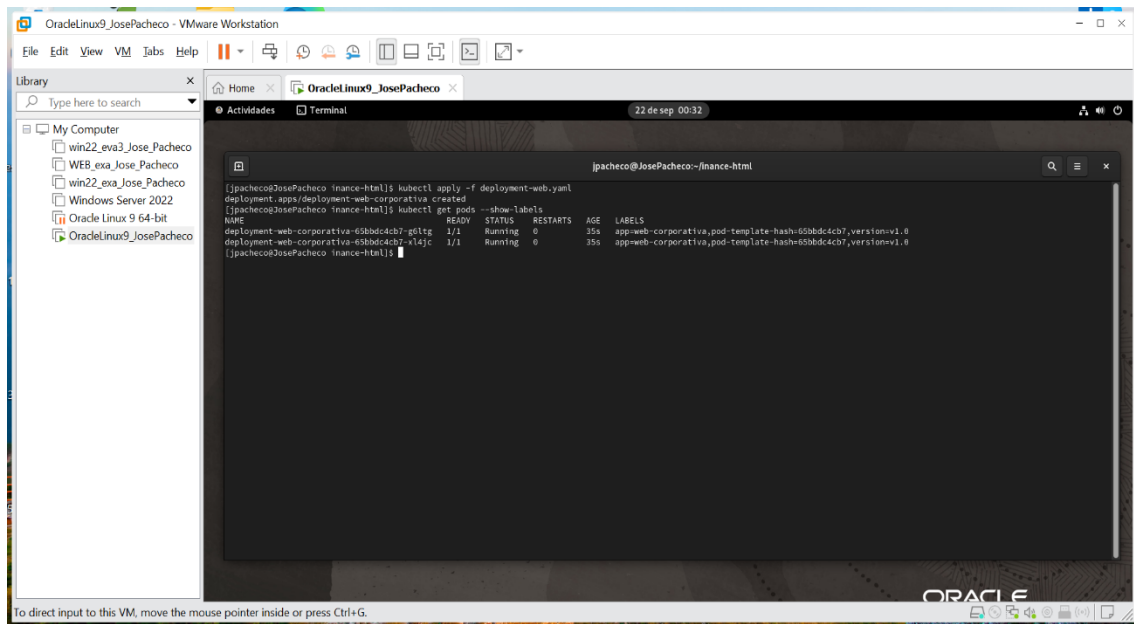
```
[jpacheco@JosePacheco ~]$ minikube start --driver=docker
minikube v1.30.0 on Oracle 9.8
* using the docker driver based on user configuration
* The requested memory allocation of 2514MiB does not leave room for system overhead (total system memory: 2514MiB). You may face stability issues.
  Suggestion: Start minikube with less memory allocated: 'minikube start --memory=2514mb'
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.1...
* Descargando kubernetes v1.37.0...
  > preloaded-images-k8s-v18-v1... 347.58 MiB / 347.58 MiB 100.00% 44.57 M
  > gcr.io/k8s-minikube/k8s:1.37.0 347.58 MiB / 347.58 MiB 100.00% 20.01 MiB
* Creating docker container (CPU=2, Memory=2514MiB) ...
* Preparando Kubernetes v1.37.0 en containerd 2.3.4...
* Configurando CNI...
* Verifying Kubernetes components...
  * Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Complementos habilitados: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
[jpacheco@JosePacheco ~]$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
minikube             Ready    control-plane  20s   v1.37.0
[jpacheco@JosePacheco ~]$
```

Para enviar la declaración de infraestructura al clúster y levantar las réplicas correspondientes, se ejecutó la orden de despliegue.

`kubectl apply -f deployment-web.yaml`

se verificó el estado de los componentes desplegados y sus etiquetas con el comando.

kubectl get pods --show-labels

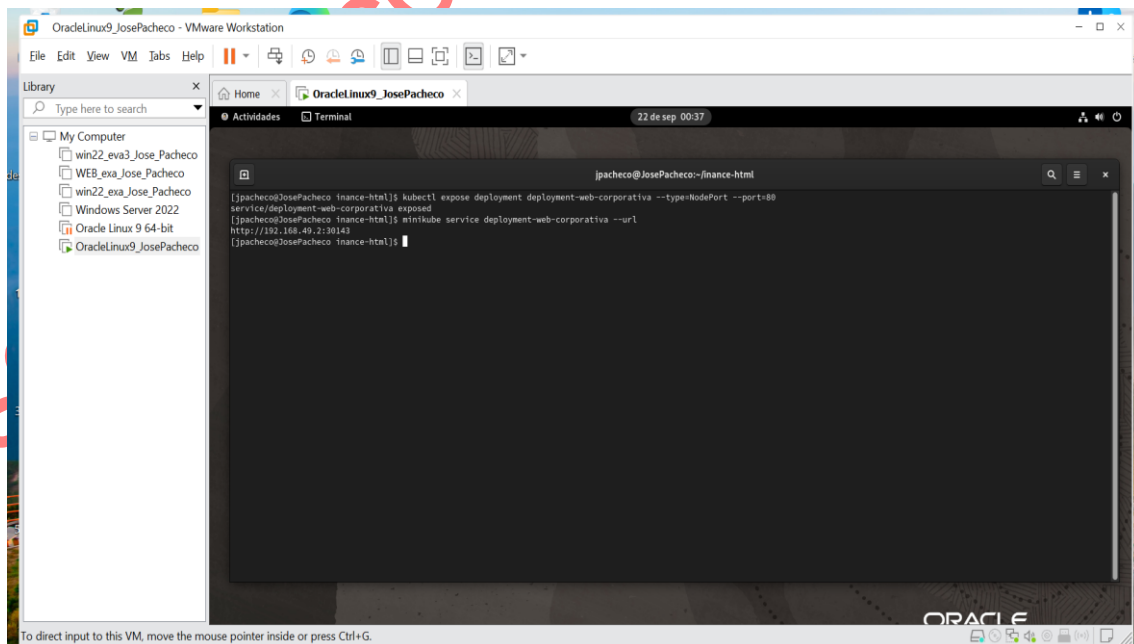


Para permitir el acceso externo al despliegue dentro del clúster, se procedió a exponer el servicio mediante la creación de un objeto de tipo NodePort.

kubectl expose deployment deployment-web-corporativa --type=NodePort --port=80

se consultó el punto de acceso mediante la utilidad de Minikube.

minikube service deployment-web-corporativa --url



Para validar la disponibilidad y el renderizado correcto de la aplicación expuesta, se ingresó la IP y puerto asignados por Minikube en la barra de direcciones del navegador Firefox.

